# Using LLMs and Generative AI for Economics Research

Hiro Kasahara

November 8, 2026

Vancouver School of Economics, University of British Columbia

# Experience and motivation

# Jan-Feb 2026: a research cycle

1. **Deep research** (ChatGPT / Gemini): retrieve ideas, aimed at top-5 journals, then generate actionable plan.
2. Feed the plan into Claude Code and run "research-cycle skills".

Applied 20+ projects. Strong in mathematical derivations and coding, but **unsupervised research cycle produces a messy paper** (notations, style, unfocused contribution).

# The auto-generated drafts were bad

Left unattended, the output fails in predictable ways:

(a) Hallucinations in the literature discussion: plausible citations that do not say what is claimed.

(b) Cited theoretical results are sometimes wrong, so propositions rest on incorrect foundations.

(c) Manuscripts come out over-complicated and unfocused: narrative, notation, and style need rewriting from scratch.

# Recent empirical evidence

- **Fabricates citations.** GPT-5.5: only 48.1% is fully accurate (Topaz et al., 2026). Details
- **Checks correctness, not contribution.** +1 SD LLM score $\Rightarrow$ 44% more likely published, driven by correctness and exposition (Lu et al., 2026). Details
- **Impacts on judgment.** Analysts followed a coin-flipped agent suggestion 86% of the time (Herdağdelen et al., 2026). Details
- **Autonomous papers errors** 842 of 1,000 had at least one severe issue (APE, 2026). Details

# Models and my setup

# The model landscape

- **Claude Code Fable/Opus**: orchestration, code, writing.
- **Codex Astra/Sol**: math, computation/simulation, code.
- **Gemini 4 Argon**: literature, full-paper synthesis, writing.
- **DeepSeek V4.1-Flash-1017**: low-cost alternative for cross-checking math/code and offloading routine tasks.
- Other options: **Grok, Muse, Kimi** etc.

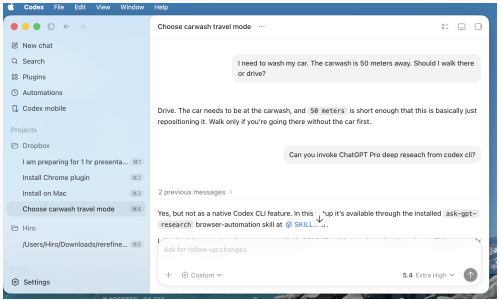
**Use all of them!**

- Assign models based on their individual strengths.
- Cross-family adversary review

# Two ways to run them

## (a) Desktop / IDE apps

- Native desktop apps, VS Code, Cursor.
- Many say better than CLI.




## (b) Terminal-based CLIs

- Claude Code CLI, Codex CLI, Antigravity CLI
- Installation is easy: ask claude.ai to get shell commands to terminal

TERMINAL

```
npm install -g @anthropic-ai/claude-code
```

```
cd ~/your-project-folder
claude
```

# My Setup: Orchestration via Subagents

- Terminal: Ghostty + Claude Code $200, Codex $100, Gemini $100, DeepSeek API.
- **Claude Code as the orchestrator** that dispatches jobs to Codex, Gemini, and DeepSeek via subagents.
- Opus5.5/xhigh with Fable5.1/xhigh as an advisor, GPT6.1-Sol/xhigh, Gemini4 Argon/high, DeepSeekV4.1-Flash-1017/max
- Macbook Pro (96GB RAM).

# CLAUDE.md/AGENTS.md and Skills

Two of the most important things when using Claude Code/Codex/Gemini:

- CLAUDE.md/AGENTS.md
- Skills

Other things worth knowing:

- settings.json
- Plugins / MCP (Model Context Protocol) servers
- hooks

# My CLAUDE.md

Read at the start of every session. Survives every `/clear`.

- **Principles**: plan, ask don't guess
- **Orchestration**: Claude Code: plan, dispatch, gate. Work goes to Codex/Gemini/DeepSeek.
- **Cross-family adversary review**: no family (Claude, GPT, Gemini, DeepSeek) reviews its own work.
- **Source-verification rule**: the defense against hallucinated citations.
- **Correctness** > **completeness** > **Claude tokens**

`CLAUDE.md` affects performance and token consumptions.

# Suggestions

# Always use “Plan mode” with a mandatory review by Codex

**Claude Code Plan ⇒ Codex Review ⇒ Implementation**

- Shift+Tab ⇒ choose "plan mode"
- Use `delegate` skill for Codex review ("hook").

Why “plan mode” with Codex review?

- Quality ↑
- Token savings by having less errors in implementation

# Writing/Slides: provide examples and styles

To write a draft or presentation slides, give exampless:

- Previous related paper or slides of yours as the style target.
- Cite related published papers for **notation and style** conventions.
- Generate your own writing-style skill

Supplying concrete examples is the high-return, low effort intervention.

# Revision of your paper

Do *not* ask directly "revise my draft." Instead:

1. Focus on one issue at a time.
2. Generate a `tex` memo for a revision plan
3. Frequent discussions with co-authors.

Key trade-offs:

- **automation** $\longleftrightarrow$ **understanding**

*"You can outsource your thinking but not your understanding."*

*- Andrej Karpathy*

# Generating presentation slides to deepen understanding

- Once I have a draft or a set of empirical results, I ask:

  "Generate slides for a 1.5-hour seminar: 40 main slides plus 20 backup slides, with cross-references back and forth."

- It forces you to understand every punchline of the paper.
- Help evaluating what else needs doing and what should be dropped.

# Revision of Your Paper by LLMs

When I ask LLMs to revise my paper, I often ask them to use ~~strikeout~~ for deletions, blue text for additions, and **bolded notes** to flag unsettled issues.

**Example**

~~Unconditional DiD~~ The transition estimator under conditional parallel trends identifies the ATT. **[Check clustered bootstrap SEs in the main table.]**

# Hallucinated citations

The model citing from memory $\Rightarrow$ fabricated references

- Generate a *Reference* folder in which all cited papers are saved as markdown files.
- Check against the original source text.

How to implement:

- `CLAUDE.md`
- Skills for citation checks and converting pdf to md files.

**Check the original sources yourself!!!**

**Watch out for copyright issues $\rightarrow$ working papers**

# Verification for theory: proofs and derivations

- Cross-validate each derivation across multiple *independent* agents (Claude Code, Codex, Gemini, DeepSeek). Enforced via `CLAUDE.md`.
- Multiple skills for checking proofs:
  - `math-verification` (proofs)
  - `lean-proof-audit` (proofs)
  - `wolfram-verify` (proofs)
- Re-derive from first principles in a fresh context (`proof-from-first-principles`) so the checker is not the author.

# Code verification: independent agent review

- **Specification memo.** Write a LaTeX spec of the models, math, algorithms, and estimation steps in details before coding.
- **Cross-family audit.** Never self-audit: if Claude writes the code, review it with Codex (high/xhigh) or DeepSeek in a fresh context (enforced via `CLAUDE.md`).
- **Paper–code consistency.** Verify the code matches the exact specification.
- **Reverse-spec validation.** In a fresh context, have an agent translate the code back into a LaTeX summary and compare it against the original spec.

# The binding constraint: performance and tokens

# Context Management and Tokens

1. **Usage limits (weekly)**




2. **Context window vs. performance.** It holds the entire conversation, every file read, every command output. Performance degrades as it fills.

1. skills
2. `/clear`
3. plan mode
4. "Bash is all you need"

# Examples of my skills

**The skills are something you create for repeated use.**

1. `delegate` (codex/antigravity/DeepSeek)
2. `gemini-pdf` (convert pdf to md files)
3. `pdf-to-reference-skill` (convert pdf to md files using Mathpix)
4. `citation-check` (hallucinations)
5. `research-vault-routing` (reference library)
8. `lean-proof-audit` (proofs)
9. `proof-from-first-principles` (proofs/hullcinations)
10. `style-guide` (writing style template)
11. `alliance-canada` (simulation/estimation)

# How to generate a skill

- Ask the model to generate a skill.

  "Generate a /delegate skill which calls Codex, Antigravity, and DeepSeek via OpenRouter from Claude Code using subagents..."

- A folder under `~/.claude/skills/`: a `SKILL.md` holding a short description plus the procedure, with optional bash/python scripts the model runs.

# Token discipline: “Bash is all you need” with skills

**Tokens are spent reading and writing in the context window. Avoid spending them on things a script can do for free.**

- Zero-token execution: let the model run a shell command and read three lines of output, not paste a whole file.
- Skills contain shell commands, python scripts etc.
- e.g., `delegate` skill.

# Memory and context management

Context is your fundamental resource.

- `/clear` once the 2M context window passes $\sim$ 60%.
- Performance degrades as the context window filled.
- 5m (subagent) or 1h (main conversation) of no prompt $\Rightarrow$ the *entire* context need to be re-processed
- Before clearing, record *what to do next*:

  tasks/todo.md, memory.md, or `claude-mem` plugin

# Infrastructure and reproducibility

# Git and GitHub

- You don't need to know much about Git and Github — Claude Code or Codex can take care of it.
- Useful for:
  - Co-authored projects
  - Homepage: `https://hkasahar.github.io/`

    "Build a GitHub Pages academic homepage in this repo using the al-folio Jekyll theme..."

# Replication packages

"Prepare an AER replication package per current AEA Data Editor policy and the Social Science Data Editors README template (fetch both first): completed README, .... Verify on a clean run."

- Languages: **R** (default), **Stata**, **Python**, **Matlab**.
- Github repos:
  - Replication Package:

    e.g., `https://github.com/hkasahar/kasahara-zhou-jhe-famine`
  - R-package: e.g., `https://github.com/bayesiahn/ak`

# Using Computer Clusters

- Submitting jobs to clusters is really easy with Claude Code.
- An example prompt:

  "Create a skill named aws-spot for submitting simulation jobs to AWS EC spot instances while minimizing the cost...."

  - `aws-spot`: AWS Spot instances / Batch for long, parallel runs.
  - `alliance-canada`: the HPC cluster in Canada for large jobs.

# Other thoughts

# Revision tips using agentic coding

- It is easy to add more analysis using LLMs. Don't!
- Focus on one key contribution.
- A shorter, clear paper will be preferred than a longer, hard-to-read paper in the era of LLMs.

**Cutting rather than adding!**

# Narrative vs. Details

**AI-written drafts are not well-written for human reading.**

- Too much details.
- Too careful for overstating.
- Weak in narratives.

$\Rightarrow$ **Fully rewrite AI-written drafts yourself.**

- **Print out the draft, read it with pen, revise, and then revise again!**

# AI Reviewers: Refine.ink, Coase.ink, Google Deep Review

- Very useful at the final stage.
- Catch technical errors well.
- But judging contribution is weak.

**Do not blindly take all issues raised by AI reviewers.**

**Take only what you think is important.**

# How to do research better in the era of LLMs

- Print out a paper to read
- Write your thoughts with pen and paper
- Discuss with co-authors frequently
- Present your paper as much as possible to get feedback
- Avoid over-automation
- Chatting with AIs rather than agentic coding

"You can outsource your thinking but not your understanding."

# References


- **Buchanan, J. A., S. Hill, and O. Shapoval** (2026): "LLM Hallucination of Citations in Economics Persists With Web-Enabled Models," *Southern Economic Journal*, 93(2), 760–765.
- **Herdağdelen, A., H. Joren, and A. Tamkin** (2026): "How AI Agents Shape Experts' Judgment Calls," *Anthropic Research*, 27 October 2026.
- **Lu, H., Y. Luo, and L. Yu** (2026): "How Well Do LLMs Perform in Peer Review? Evidence from Finance and Accounting," *arXiv preprint arXiv:2610.18435*, 13 October 2026.
- **Project APE** (2026): "Evaluating the First 1,000 APE Papers" and "Does Paper Complexity Increase Errors?" *Social Catalyst Lab, University of Zurich*, 13 October 2026.
- **Topaz, M., Z. Zhang, N. Roguin, P. Gupta, Z. Li, and L.-M. Peltonen** (2026): "Biomedical Reference Generation Remains Unreliable across 26 Large Language Models," *arXiv preprint arXiv:2609.14988*, 14 September 2026.

# Fabricated citations persist in web-enabled models

- **Web search does not cure hallucination.** Web-enabled GPT-5.1 and GPT-4o still invent non-existent economics citations across JEL prompts (Buchanan et al., 2026).
- **26-LLM biomedical benchmark.** 55.4% of references fabricated overall; only 14.9% accurate in every field (Topaz et al., 2026).
- **2026 frontier models.** 35.3% fabricated and 31.8% fully accurate (best model, GPT-5.5: 48.1%; Topaz et al., 2026).
- **Metadata corruption.** Real papers often have corrupted authors, years, volumes, or pages.

Back

# Checks correctness, not contribution

- **Predicts top-3 finance publication.** Across 276 papers, +1 SD LLM score $\Rightarrow$ 44% higher publication probability in *JF/JFE/RFS* (+2.9 pp on 6.5% base rate, $z = 2.39$; Lu et al., 2026).
- **Robust signal.** Holds net of author institution, track record, and editorial history; replicates in accounting and with GPT-5.
- **Driven by mechanics.** Predictive power comes entirely from correctness and exposition.
- **Blind to importance.** Zero signal on substantive contribution, even when given the evaluation rubric.

Back

# Agents steer expert judgment calls

- **Anthropic RCT.** 50 professional analysts used Claude Code (Opus 4.8, high effort) across four open analytical choices (Herdağdelen et al., 2026).
- **86% compliance with coin flips.** Analysts followed the agent's random lean 86% of the time (81%–92% across choices; 72 pp pooled contrast).
- **Overrides prior views.** When the agent disagreed with an analyst's initial preference, 60% switched (vs. 6% when it agreed).
- **Unacknowledged influence.** 77% repeated the agent's rationale as their own; only 15% credited the agent; expertise did not reduce deferral.

Back

# Autonomous papers err at scale

- **1,000-paper audit.** Project APE audited 1,000 autonomous policy papers using Codex 5.5 (high) under the CRED v0.92 taxonomy (APE, 2026).
- **Widespread severe defects.** Only 144 of 1,000 reproduced tables cleanly; 842 of 1,000 had at least one severe issue (4.3–4.43 defects/paper).
- **Paper–code divergence.** Doubling text length adds 0.82 paper–code inconsistencies; doubling R code lines adds 0.52.
- **Strong auditor.** Human reviewers confirmed 97.4% of Codex 5.5 (high) findings (96.5% recall on injected errors).

Back